

HikeAlong

Version: 1.0

Vision

Date: 25/May/26

Document Identifier: HikeAlong-Vision-1.0

Raisa Butuza

Group 30431

Revision History

Date	Version	Description	Author
25/May/26	1.0	Initial version of the Vision document for the HikeAlong event reservation system.	Raisa Butuza

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	5
1.3	Definitions, Acronyms, and Abbreviations	5
1.4	References	6
1.5	Overview	6
2	Positioning	7
2.1	Problem Statement	7
2.2	Product Position Statement	7
3	Stakeholder and User Descriptions	8
3.1	Stakeholder Summary	8
3.2	User Summary	9
3.3	User Environment	10
4	Product Requirements	11
4.1	Functional Requirements	11
4.2	UI/UX Requirements	16
4.3	Backend Requirements	19
4.4	Frontend Requirements	21
4.5	Testing Requirements	23
4.6	Performance Requirements	24
4.7	Security Requirements	25
4.8	Availability Requirements	25
4.9	Maintainability Requirements	26
4.10	Demo Requirements	26
4.11	Product Constraints	27

5 Conclusion**28**

1. Introduction

The purpose of this document is to collect, analyze, and define the high-level needs and features of HikeAlong. The document focuses on what the application must offer to its stakeholders and users, why the system is needed, and what value it brings.

HikeAlong is a full-stack web application for managing hiking events and event reservations. It allows users to browse available events, search and filter events, reserve seats, and receive reservation updates. It also allows organizers to create and manage their own events, view reservation requests, and accept or reject reservations.

The application is built using a Spring Boot backend and an Angular frontend. The backend exposes REST endpoints for authentication, users, events, and reservations. The frontend offers a role-based interface, where users and organizers see different pages and actions depending on their role.

The project also includes bonus features that improve the final user experience:

- real-time WebSocket notifications for reservation status updates;
- toast notifications in the frontend;
- role-based toolbar and navigation;
- event search, filtering, and sorting;
- clean UI using Angular Material components;
- user-friendly validation and error messages.

1.1 Purpose

The purpose of this Vision document is to describe the main goals, scope, users, stakeholders, and high-level product requirements of the HikeAlong application.

This document is intended to guide the development of the system and to provide a clear understanding of what the application should achieve. It does not describe the complete technical implementation in detail. Instead, it explains the system from a product and requirements perspective.

The detailed technical requirements, quality attributes, and design constraints are described separately in the Supplementary Specification document.

1.2 Scope

The scope of this document covers the complete HikeAlong application, including:

- backend functionality implemented with Spring Boot;
- frontend functionality implemented with Angular;
- authentication and login flow;
- role-based access for USER and ORGANIZER accounts;
- Person, Event, and Reservation entity management;
- reservation creation and reservation status updates;
- event searching, filtering, and sorting;
- WebSocket notifications for real-time reservation updates;
- UI/UX features used during the live demonstration;
- backend testing requirements;
- basic deployment and demo environment requirements.

The project is associated with the Software Design assignment and is intended to demonstrate a complete full-stack application with backend, frontend, documentation, testing, authentication, and role-based functionality.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
HikeAlong	The name of the application. It is a web platform for hiking event management and reservation.
User	A person who uses the application to browse events and make reservations.
Organizer	A person who creates and manages hiking events and handles reservation requests.
Person	The backend entity that stores user account information, including name, email, password, age, and role.

Event	The backend entity that stores information about a hiking event, including title, description, location, date, maximum participants, and organizer.
Reservation	The backend entity that connects a person to an event and stores the number of reserved spots and the reservation status.
CRUD	Create, Read, Update, Delete. These are the basic operations supported for core entities.
REST API	An HTTP-based API used by the frontend to communicate with the backend.
DTO	Data Transfer Object. A class used to transfer and validate data between the frontend and backend.
JPA	Java Persistence API. Used together with Spring Data JPA for database persistence.
WebSocket	A communication protocol used for real-time notifications between the backend and frontend.
Toast Notification	A small frontend message that appears after an action, such as a success, error, or reservation status update.
UI/UX	User Interface and User Experience. It refers to how the application looks and how easy it is to use.

1.4 References

The following documents and project artifacts are referenced by this Vision document:

- Software Design Assignment 1 documentation for the event reservation system.
- Software Design Assignment 2 requirements.
- Project Supplementary Specification document.
- Backend source code for controllers, entities, DTOs, validation, and exception handling.
- Frontend Angular implementation for login, event feed, reservation pages, organizer pages, and notification UI.

1.5 Overview

The rest of this document is organized as follows:

- Section 2 presents the positioning of the product, including the problem statement and product position statement.
- Section 3 describes the stakeholders, users, and user environment.
- Section 4 lists the main product requirements at a high level.

2. Positioning

2.1 Problem Statement

The problem of	Managing hiking events and reservations manually or through disconnected tools is inefficient, error-prone, and difficult to follow for both users and organizers.
Affects	Basic users who want to discover and reserve hiking events, organizers who need to manage events and reservation requests, and evaluators who need to see a complete full-stack application with clear flows.
The impact of which is	Users may not easily find suitable events, organizers may have difficulty tracking reservations, and reservation status updates may not be communicated clearly or quickly. Manual management also increases the risk of duplicate reservations, invalid data, and inconsistent event capacity.
A successful solution would be	A web application where users can browse, search, filter, and reserve events, while organizers can manage their events and reservation requests. The system should provide clear validation, role-based access, real-time reservation notifications, and a clean frontend interface for live demonstration.

2.2 Product Position Statement

For	Students, hiking enthusiasts, event participants, and event organizers.
Who	Need a simple and structured way to create, discover, reserve, and manage hiking events.
The HikeAlong application	Is a full-stack event reservation web application.

That	Provides event browsing, role-based event management, reservation creation, reservation status updates, search/filter/sort functionality, validation, authentication, and real-time notifications.
Unlike	Manual spreadsheets, chat groups, or static event lists that do not provide structured reservations, role-based access, or real-time updates.
Our product	Combines a Spring Boot backend, Angular frontend, REST API communication, WebSocket notifications, and UI/UX improvements into one coherent event reservation platform.

3. Stakeholder and User Descriptions

This section describes the main stakeholders and users of the HikeAlong application. A user is a person who directly interacts with the system, while a stakeholder is a person or group that has an interest in the success, quality, maintainability, or evaluation of the system.

3.1 Stakeholder Summary

Name	Description	Responsibilities
Project Developer	The student or development team responsible for implementing the application.	Designs, implements, tests, documents, and presents the backend and frontend application. Ensures that the project respects the assignment requirements and coding standards.
Course Evaluator	The teacher or laboratory evaluator who checks the project.	Evaluates the backend, frontend, documentation, tests, authentication flow, role-based behavior, and live demo scenarios. Checks if the application satisfies the Software Design assignment requirements.

Application Main-tainer	A future developer who may continue or improve the application.	Ensures that the project remains maintainable, modular, testable, and easy to extend. Benefits from layered architecture and clear documentation.
Event Organization Stakeholder	A person or group interested in managing real hiking events through the platform.	Defines practical expectations for event creation, reservation tracking, and communication with participants.
End User Representative	A person representing the needs of basic users and event participants.	Ensures that the application is easy to use, clear, and helpful for browsing events and making reservations.
Organizer Representative	A person representing the needs of event organizers.	Ensures that organizers can easily create events, edit event information, view reservation requests, and accept or reject reservations.

3.2 User Summary

Name	Description	Responsibilities	Stakeholder
Basic User	A person who uses the application to browse hiking events and make reservations.	Logs in, views the event feed, searches and filters events, opens event details, reserves spots, views reservation status, and receives notifications.	End User Representative
Organizer	A person responsible for creating and managing hiking events.	Logs in, creates events, edits events, deletes events, views reservations for events, accepts or rejects reservations, and triggers reservation status updates.	Organizer Representative

Admin or Demo User	A user type that may be used for project demonstration or management purposes, if implemented.	May view or manage users, events, and reservations depending on the project configuration. Used to demonstrate role-based access more clearly.	Course Evaluator
Unauthenticated Visitor	A person who reaches the application before logging in.	Can access the login page and must authenticate before using role-based application functionality.	End User Representative

3.3 User Environment

The target users interact with HikeAlong through a web browser. The application is designed mainly for desktop or laptop usage during the academic project demonstration, but the interface can also be used on smaller screens if the Angular layout is responsive.

The expected user environment includes:

- a browser such as Chrome, Edge, or Firefox;
- a running Angular frontend;
- a running Spring Boot backend;
- a relational database connected to the backend;
- a stable local or network connection between frontend and backend;
- a WebSocket connection for real-time notifications, when available.

The main task cycles are short:

- login should take less than one minute;
- browsing and filtering events should take only a few seconds;
- creating a reservation should be possible in one short flow;
- accepting or rejecting a reservation should be possible from the organizer reservation page;

- receiving a notification should happen automatically after the reservation status changes.

The application is especially designed for a live demonstration. This means that the frontend must make the main flows visible and easy to understand without needing to inspect the database directly.

4. Product Requirements

This section lists the high-level product requirements of the HikeAlong application. These requirements describe the expected behavior and qualities of the whole system.

4.1 Functional Requirements

FR1. User Authentication

The system shall provide a login page where users enter their email and password.

The frontend shall send a POST request to the backend login endpoint. If the credentials are correct, the user shall be redirected based on role. If the credentials are wrong, the user shall see an error message.

- USER accounts are redirected to the event feed.
- ORGANIZER accounts are redirected to the organizer area.
- If ADMIN is used in the demo, ADMIN accounts are redirected to an admin area.

FR2. Role-Based Access

The system shall support role-based functionality.

- Basic users can browse events and create reservations.
- Organizers can manage events and reservation requests.
- The frontend toolbar shall display different navigation links based on the current role.
- Actions that do not belong to the current role should be hidden from the interface.

FR3. Person Management

The system shall support Person management.

The Person entity stores:

- unique identifier;
- name;
- email;
- password;
- age;
- role.

The system shall validate person input and prevent duplicate email addresses.

FR4. Event Management

The system shall support Event management.

The Event entity stores:

- unique identifier;
- title;
- description;
- location;
- date;
- maximum number of participants;
- organizer.

Organizers shall be able to:

- create events;
- view their events;
- update events;
- delete events.

FR5. Event Browsing

Basic users shall be able to browse available hiking events from the event feed.

The event feed shall display the most important event information:

- event title;
- description;
- location;
- date;
- maximum participants or availability;
- organizer information, if needed;
- reservation action.

FR6. Event Search, Filtering, and Sorting

The Event entity shall support searching, filtering, and sorting.

The event feed shall support:

- search by title;
- filtering by location;
- filtering by date or upcoming events;
- sorting by date;
- optionally sorting by available places or maximum number of participants.

This feature is important because events are the main entity browsed by users. It improves the usability of the application and also satisfies the assignment requirement for search, ordering, and filtering.

FR7. Reservation Management

The system shall support Reservation management.

The Reservation entity connects a Person to an Event and stores:

- unique identifier;
- person;
- event;
- number of reserved spots;
- reservation status.

The reservation status can be:

- PENDING;
- ACCEPTED;
- DECLINED.

Basic users shall be able to create reservations. Organizers shall be able to view reservation requests and change their status.

FR8. Reservation Status Update

The organizer shall be able to accept or reject a reservation.

When the reservation status is updated:

- the backend saves the new status;
- the frontend displays a success message to the organizer;
- the user can see the updated reservation status;
- the WebSocket notification system may send a real-time message to the user.

FR9. WebSocket Notifications

The system shall include a WebSocket notification feature as a bonus part.

The notification flow is:

1. A user creates a reservation.
2. The reservation is stored as PENDING.

3. The organizer accepts or rejects the reservation.
4. The backend updates the reservation status.
5. The backend sends a WebSocket message.
6. The frontend receives the message.
7. The frontend displays a toast notification.

Example notification messages:

- Your reservation was accepted.
- Your reservation was rejected.
- A reservation status has been updated.

The WebSocket feature should not replace the REST API. It is used only to improve the user experience with real-time updates.

FR10. Frontend Toast Notifications

The frontend shall provide toast notifications for important actions.

Toast notifications should appear when:

- login fails;
- an event is created successfully;
- an event is updated or deleted;
- a reservation is created;
- a reservation is accepted or rejected;
- a WebSocket notification is received;
- a validation or backend error occurs.

FR11. Profile Page

The system shall provide a profile page where users can view and update their profile information.

The profile page may support:

- displaying name, email, age, and role;
- editing name, email, age, or password;
- showing validation errors;
- saving updates through the backend API.

FR12. Error Handling

The backend shall return meaningful error responses for invalid operations.

The frontend shall display these errors in a user-friendly way.

Examples of errors:

- invalid email format;
- weak password;
- duplicate email;
- invalid event date;
- invalid number of reserved spots;
- wrong login credentials;
- reservation already exists;
- unauthorized role action.

4.2 UI/UX Requirements

The frontend shall be clear, structured, and easy to use during the live demonstration.

UX1. Role-Based Toolbar

The toolbar shall display navigation links depending on the current role.

Example:

- USER: Events, Reservations, Profile.
- ORGANIZER: Events, My Events, Reservations, Profile.
- ADMIN, if used: Manage Users, Manage Events, Manage Reservations.

UX2. Event Feed Design

The event feed shall present events in a readable way.

The feed should include:

- event cards or clean list items;
- visible title, location, date, and capacity;
- a clear reservation button for users;
- no organizer-only actions for basic users;
- search and filter controls at the top.

UX3. Organizer Dashboard

The organizer area shall make event management simple.

The organizer should be able to:

- view own events;
- create a new event;
- edit event details;
- delete an event;
- access reservations related to their events.

UX4. Reservation Management Interface

The organizer reservation page shall show reservation requests clearly.

The page should include:

- reservation user information;
- event information;
- number of reserved spots;
- current reservation status;
- Accept and Reject buttons;
- visual feedback after status update.

UX5. Validation Feedback

Forms shall guide the user when data is invalid.

Examples:

- show that email must be valid;
- show that password must be strong;
- show that event date must be in the future;
- show that reserved spots must be at least 1;
- show backend error messages when the request fails.

UX6. Notification Experience

Real-time and normal notifications shall be displayed in a small, non-blocking toast component.

The notification component should:

- appear after important actions;
- disappear automatically after a short time;
- use clear messages;
- avoid interrupting the main user flow;
- show reservation status updates received through WebSocket.

4.3 Backend Requirements

BR1. Layered Architecture

The backend shall follow a layered architecture:

- Controller layer for REST endpoints;
- Service layer for business logic;
- Repository layer for data access;
- Model layer for entities and DTOs;
- Configuration layer for validation and exception handling;
- Notification layer for WebSocket messages.

BR2. REST Controllers

The backend shall expose REST endpoints for:

- login;
- persons;
- events;
- reservations.

The main endpoints are:

Endpoint	Purpose
POST /login	Authenticates a user and returns the login result and role.
/person	Handles person CRUD operations.
/event	Handles event CRUD operations.
/reservation	Handles reservation CRUD operations and status updates.

BR3. Business Logic

The service layer shall enforce the main business rules.

Examples:

- prevent duplicate emails;

- validate person data;
- validate event data;
- check that event dates are in the future;
- check that reservation spots are valid;
- prevent duplicate reservations for the same person and event, if required;
- update reservation status correctly;
- send notifications after reservation status changes.

BR4. Persistence

The backend shall use Spring Data JPA for database access.

The main relationships are:

- one organizer can create many events;
- one user can have many reservations;
- one event can have many reservations;
- Person and Event are connected many-to-many through Reservation.

BR5. Global Exception Handling

The backend shall use a centralized exception handler to return consistent error responses.

The exception handling should cover:

- DTO validation errors;
- custom field validation errors;
- business validation errors;
- wrong login credentials;
- invalid request data.

4.4 Frontend Requirements

FER1. Angular Pages

The frontend shall include multiple pages.

Common pages:

- Login page;
- Register page, if implemented;
- Event feed page;
- Profile page.

User pages:

- Browse Events;
- Event Details;
- My Reservations;
- Reservation creation flow.

Organizer pages:

- My Events;
- Create Event;
- Edit Event;
- Organizer Reservations;
- Reservation approval and rejection.

Admin pages, if used:

- Manage Users;
- Manage Events;
- Manage Reservations.

FER2. State Management

The frontend shall store and use the logged-in user's state.

The frontend should track:

- login success;
- current role;
- current email or user identifier;
- authentication data stored in session storage;
- current navigation state.

FER3. Frontend Services

The frontend shall use Angular services to communicate with the backend.

Examples:

- Login service;
- Person service;
- Event service;
- Reservation service;
- Notification service;
- WebSocket notification service.

FER4. WebSocket Frontend Behavior

The frontend shall connect to the backend WebSocket endpoint after login or when the application starts.

When a notification is received:

- the frontend reads the message;
- the notification service receives it;
- a toast message is displayed;
- the user understands that a reservation status changed.

4.5 Testing Requirements

The backend shall include unit tests and integration tests.

The main tested entities are:

- Person;
- Event;
- Reservation.

TR1. Person Tests

Person tests should include:

- create person successfully;
- reject duplicate email;
- reject invalid email;
- reject weak password;
- update person successfully;
- delete person successfully.

TR2. Event Tests

Event tests should include:

- create event successfully;
- reject invalid title;
- reject invalid location;
- reject past date;
- reject invalid participant number;
- update event successfully;
- delete event successfully;
- search or filter events, if implemented in backend.

TR3. Reservation Tests

Reservation tests should include:

- create reservation successfully;
- reject invalid number of spots;
- reject reservation for missing person;
- reject reservation for missing event;
- reject duplicate reservation, if this rule is active;
- update reservation status;
- delete reservation successfully;
- trigger notification after status update, if included in test scope.

TR4. Integration Tests

Controller integration tests should verify real request and response behavior.

Integration tests should cover:

- Person controller;
- Event controller;
- Reservation controller;
- login endpoint;
- validation errors;
- successful CRUD flows.

4.6 Performance Requirements

The application shall perform well for a normal academic demonstration dataset.

- Login should complete in under 2 seconds in local execution.
- Event feed loading should complete in under 2 seconds for a normal demo dataset.

- Event filtering and sorting should feel immediate on the frontend.
- Reservation creation should complete in under 2 seconds.
- Reservation status update should complete in under 2 seconds.
- WebSocket toast notifications should appear shortly after the backend sends the update.

4.7 Security Requirements

The application shall protect user data and restrict actions based on role.

- Users must log in before accessing role-based pages.
- Wrong credentials must return an error.
- The frontend shall redirect users based on role.
- The frontend shall hide unauthorized actions.
- The backend shall validate important data and business rules.
- Passwords should not be exposed in frontend responses.
- WebSocket notifications should not expose sensitive information.

4.8 Availability Requirements

The application shall remain usable during normal operation.

- REST operations should work independently from the WebSocket notification system.
- If WebSocket is disconnected, the user should still be able to browse events and make reservations.
- Reservation status updates should still be saved even if a notification cannot be delivered.
- Users should be able to see the latest status by refreshing or reopening the reservation page.

4.9 Maintainability Requirements

The application shall be easy to understand and extend.

- The backend shall keep controllers, services, repositories, models, DTOs, and configuration separated.
- The frontend shall keep components, services, stores, and models separated.
- Code should use consistent naming.
- Unused code should be removed.
- Static classes should be avoided unless clearly justified.
- The project should follow REST API design guidelines.
- Tests should make the main flows safer to modify.

4.10 Demo Requirements

The final project demonstration shall show the full application flow.

Recommended demo scenario:

1. Log in as a USER.
2. Browse the event feed.
3. Search for an event.
4. Filter events by location or date.
5. Reserve seats for an event.
6. Show the reservation as PENDING.
7. Log out.
8. Log in as an ORGANIZER.
9. Open organizer reservations.
10. Accept or reject the user's reservation.
11. Show the toast notification.

12. Return to the USER view and show the updated reservation status.

A second demo scenario can show event management:

1. Log in as an ORGANIZER.
2. Create a new hiking event.
3. Edit the event details.
4. View it in the event feed.
5. Delete the event or keep it for reservation testing.

A third demo scenario can show validation:

1. Try to create an event with a past date.
2. Show the validation error.
3. Try to reserve zero spots.
4. Show the validation error.
5. Try to log in with a wrong password.
6. Show the login error message.

4.11 Product Constraints

The project must follow the assignment constraints:

- The deliverable must include a backend application.
- The deliverable must include a frontend application.
- The deliverable must include required documentation.
- The project must be uploaded to Git before presentation.
- The project should not be modified after the deadline without penalty.
- The code should be clean, readable, and consistently formatted.
- The frontend should be demonstrable live.

5. Conclusion

The HikeAlong application is designed as a complete full-stack event reservation platform for hiking events. It covers the required academic functionality, including backend CRUD, frontend pages, login, role-based redirects, testing, validation, and documentation.

The project also includes stronger user-focused and bonus features, such as WebSocket reservation notifications, toast messages, role-based navigation, event search, filtering, sorting, and a cleaner UI/UX. These additions make the application more realistic and easier to demonstrate.

The main value of the application is that it connects users and organizers in one structured system. Users can discover and reserve events, while organizers can manage events and reservation requests efficiently. The WebSocket notification feature improves communication by giving users immediate feedback when their reservation status changes.